

# **Competitive Programming**

**Section 33 Editorial: Inclusion-Exclusion**

Competitive Programming Course

2026-07-06

# Table of contents

|          |                                |           |
|----------|--------------------------------|-----------|
| <b>1</b> | <b>Editorial overview</b>      | <b>3</b>  |
| <b>2</b> | <b>A. Multiples Union</b>      | <b>4</b>  |
| 2.1      | Restatement . . . . .          | 4         |
| 2.2      | Algorithm . . . . .            | 4         |
| 2.3      | Correctness proof . . . . .    | 4         |
| 2.4      | Complexity . . . . .           | 4         |
| 2.5      | Full C++ solution . . . . .    | 4         |
| 2.6      | Full Python solution . . . . . | 5         |
| <b>3</b> | <b>B. All Symbols</b>          | <b>7</b>  |
| 3.1      | Restatement . . . . .          | 7         |
| 3.2      | Algorithm . . . . .            | 7         |
| 3.3      | Correctness proof . . . . .    | 7         |
| 3.4      | Complexity . . . . .           | 7         |
| 3.5      | Full C++ solution . . . . .    | 7         |
| 3.6      | Full Python solution . . . . . | 8         |
| <b>4</b> | <b>C. Bounded Candies</b>      | <b>10</b> |
| 4.1      | Restatement . . . . .          | 10        |
| 4.2      | Algorithm . . . . .            | 10        |
| 4.3      | Correctness proof . . . . .    | 10        |
| 4.4      | Complexity . . . . .           | 10        |
| 4.5      | Full C++ solution . . . . .    | 10        |
| 4.6      | Full Python solution . . . . . | 11        |
| <b>5</b> | <b>Testing discussion</b>      | <b>13</b> |
| <b>6</b> | <b>What to carry forward</b>   | <b>14</b> |

# 1 Editorial overview

| Problem            | Events              | Intersection size                   |
|--------------------|---------------------|-------------------------------------|
| A. Multiples Union | divisible by $d[i]$ | $N / \text{lcm}(\text{subset})$     |
| B. All Symbols     | symbol is missing   | $(k-j)^n$                           |
| C. Bounded Candies | child exceeds bound | stars-and-bars after removing $b+1$ |

## 2 A. Multiples Union

### 2.1 Restatement

Count integers in  $[1, N]$  divisible by at least one listed divisor.

### 2.2 Algorithm

Enumerate every nonempty subset of divisors. Compute the lcm of selected divisors. If the subset size is odd, add  $N/\text{lcm}$ ; if even, subtract it.

Stop building a subset lcm once it exceeds  $N$ , because it contributes zero.

### 2.3 Correctness proof

For each divisor  $d[i]$ , define event  $E_i$  as “the number is divisible by  $d[i]$ .” A number is in the desired set exactly when it belongs to the union of these events. For any subset of events, their intersection is the set of numbers divisible by all selected divisors, i.e. divisible by the selected lcm. Thus the algorithm applies the inclusion-exclusion formula to the exact event union.  $\square$

### 2.4 Complexity

Time:  $O(m \cdot 2^m)$

Memory:  $O(1)$

### 2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    int m;
    if (!(cin >> n >> m)) return 0;
```

```

vector<long long> d(m);
for (long long &x : d) cin >> x;

long long answer = 0;
for (int mask = 1; mask < (1 << m); mask++) {
    __int128 lcm = 1;
    int bits = 0;
    bool ok = true;
    for (int i = 0; i < m; i++) {
        if (mask & (1 << i)) {
            bits++;
            long long g = gcd((long long)min<__int128>(lcm, LLONG_MAX), d[i]);
            lcm = lcm / g * d[i];
            if (lcm > n) {
                ok = false;
                break;
            }
        }
    }
    if (!ok) continue;
    long long count = n / (long long)lcm;
    if (bits % 2) answer += count;
    else answer -= count;
}
cout << answer << '\n';
return 0;
}

```

## 2.6 Full Python solution

```

import math
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m = data[0], data[1]
    d = data[2 : 2 + m]
    answer = 0
    for mask in range(1, 1 << m):
        lcm = 1
        bits = 0
        ok = True
        for i in range(m):
            if (mask >> i) & 1:
                bits += 1

```

```
        g = math.gcd(lcm, d[i])
        lcm = lcm // g * d[i]
        if lcm > n:
            ok = False
            break
    if ok:
        count = n // lcm
        answer += count if bits % 2 == 1 else -count
print(answer)
```

## 3 B. All Symbols

### 3.1 Restatement

Count strings of length  $n$  over  $k$  symbols where every symbol appears at least once.

### 3.2 Algorithm

Let a bad event mean “this symbol is missing.” If  $j$  symbols are missing, the string uses only  $k-j$  symbols, giving  $(k-j)^n$  strings. Choose the missing symbols in  $C(k, j)$  ways.

```
answer = sum_j (-1)^j C(k,j) (k-j)^n
```

### 3.3 Correctness proof

We start from all strings and exclude the union of bad events. Inclusion- exclusion over missing-symbol events subtracts strings missing at least one symbol and restores those missing multiple symbols the correct number of times. The intersection of any  $j$  missing-symbol events has  $(k-j)^n$  strings, and there are  $C(k, j)$  such intersections.  $\square$

### 3.4 Complexity

```
Time:  $O(k)$  per query
```

### 3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
}
```

```

    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<pair<long long, int>> queries(q);
    int limit = 0;
    for (auto &[n, k] : queries) {
        cin >> n >> k;
        limit = max(limit, k);
    }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[limit] = mod_pow(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
    auto C = [&](int n, int r) -> long long {
        if (r < 0 || r > n) return 0;
        return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
    };
    for (auto [n, k] : queries) {
        long long ans = 0;
        for (int missing = 0; missing <= k; missing++) {
            long long term = C(k, missing) * mod_pow(k - missing, n) % MOD;
            if (missing % 2) ans = (ans - term + MOD) % MOD;
            else ans = (ans + term) % MOD;
        }
        cout << ans << '\n';
    }
    return 0;
}

```

## 3.6 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)

```

```

for i in range(limit, 0, -1):
    invfact[i - 1] = invfact[i] * i % MOD
return fact, invfact

def comb(n: int, r: int, fact: list[int], invfact: list[int]) -> int:
    if r < 0 or r > n:
        return 0
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    queries = [(data[i], data[i + 1]) for i in range(1, 2 * q + 1, 2)]
    fact, invfact = prepare(max(k for _, k in queries))
    out = []
    for n, k in queries:
        ans = 0
        for missing in range(k + 1):
            term = comb(k, missing, fact, invfact) * pow(k - missing, n, MOD) % MOD
            if missing % 2:
                ans -= term
            else:
                ans += term
        out.append(str(ans % MOD))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

## 4 C. Bounded Candies

### 4.1 Restatement

Count nonnegative solutions to  $x_1 + \dots + x_n = s$  with every  $x_i \leq b$ .

### 4.2 Algorithm

Bad event: a selected child receives at least  $b+1$  candies. For  $j$  bad children, reserve  $b+1$  candies for each, then distribute the remaining candies freely among all children:

$$C(s - j(b+1) + n - 1, n - 1)$$

Choose the bad children in  $C(n, j)$  ways and alternate signs.

### 4.3 Correctness proof

A distribution violating exactly  $t$  upper bounds is included in the term for each nonempty subset of those  $t$  violated children. The alternating sum over subsets counts invalid distributions as zero and valid distributions as one. The stars-and-bars term counts the intersection size after forcing chosen children to exceed the bound.  $\square$

### 4.4 Complexity

Time:  $O(\min(n, s/(b+1)))$  per query

### 4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
```

```

        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<array<int, 3>> queries(q);
    int limit = 0;
    for (auto &cur : queries) {
        cin >> cur[0] >> cur[1] >> cur[2];
        limit = max(limit, cur[0] + cur[1]);
    }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[limit] = mod_pow(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
    auto C = [&](int n, int r) -> long long {
        if (r < 0 || r > n) return 0;
        return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
    };
    for (auto [n, s, b] : queries) {
        long long ans = 0;
        int step = b + 1;
        for (int bad = 0; bad <= n && bad * step <= s; bad++) {
            int remaining = s - bad * step;
            long long term = C(n, bad) * C(remaining + n - 1, n - 1) % MOD;
            if (bad % 2) ans = (ans - term + MOD) % MOD;
            else ans = (ans + term) % MOD;
        }
        cout << ans << '\n';
    }
    return 0;
}

```

## 4.6 Full Python solution

```

import sys

MOD = 1_000_000_007

```

```

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

def comb(n: int, r: int, fact: list[int], invfact: list[int]) -> int:
    if r < 0 or r > n:
        return 0
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    queries = [(data[i], data[i + 1], data[i + 2]) for i in range(1, 3 * q + 1, 3)]
    limit = max(n + s for n, s, _ in queries)
    fact, invfact = prepare(limit)
    out = []
    for n, s, b in queries:
        ans = 0
        step = b + 1
        for bad in range(0, min(n, s // step) + 1):
            remaining = s - bad * step
            term = comb(n, bad, fact, invfact) * comb(remaining + n - 1, n - 1, fact, invfact)
            ans += term if bad % 2 == 0 else -term
        out.append(str(ans % MOD))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

## 5 Testing discussion

The random tests brute-force small cases:

```
Multiples Union: scan every  $x \leq N$ .  
All Symbols: enumerate all short strings.  
Bounded Candies: enumerate all bounded vectors.
```

This is especially useful for inclusion-exclusion because a single sign error often passes simple samples.

## 6 What to carry forward

Inclusion-exclusion is a counting correction tool. Define events clearly, derive the size of an intersection, then apply the sign pattern based on how many events are selected.